

webアプリ制作 機能一覧（バックエンド）

1. 認証・ユーザー管理機能

アプリケーションを利用するための基本的なアカウント管理機能です。

- **ユーザー登録機能:** ユーザID とパスワード を受け取り、データベースに保存して新規会員登録を行う処理。
- **ログイン・認証機能:** ユーザID とパスワード の照合を行い、ログイン（サインイン）を許可する処理。
- **ユーザー検索機能:** イベントの送信先を選択 するために、登録されているユーザで検索を行う機能。
- **送信履歴取得機能:** ユーザーの過去の利用履歴から、過去に送信した人 を一覧として取得する機能。

2. 幹事向け機能（イベント管理）

「幹事メニュー」に関連する、イベントの作成から集計・確定までの機能です。

- **管理イベント一覧取得機能:** 自身が幹事として作成した管理イベント一覧 を取得する機能。
- **イベントステータス管理機能:** 各イベントの状態が検討中 なのか開催済 なのかを管理・更新する機能。
- **イベント新規作成機能:** 日付選択機能（カレンダー） や、エリア、予算 などの初期条件を受け取り、イベントを新規作成する機能。
- **招待送信機能:** 検索した特定のユーザー（例: MasudaShuzo など）を送信先一覧 としてデータベース上でイベントに紐付け、招待を送信する機能。
- **回答集計機能:** 参加者からの回答をバックエンドで計算し、日程一覧ごとの点数（例: 5点、8点など）、希望予算、希望エリア、および全体コメント を集計してフロントエンドに返す機能。
- **イベント確定機能:** 最終的な日時、予算、エリア、店名、支払先や支払QRコードリンク などの確定内容 を保存し、参加者に送信・通知する機能。

3. 参加者向け機能（回答管理）

「参加者メニュー」に関連する、招待の確認から回答・確定情報の閲覧までの機能です。

- **参加イベント一覧取得機能:** 自身が招待されている参加イベント一覧 を取得する機能。
- **イベントフィルタリング機能:** 一覧表示の際、自身の回答状況に応じて未回答 と回答済 でイベントを絞り込む機能。
- **イベント詳細取得機能:** 指定したイベントの詳細（イベント名、メ切、幹事名など） を取得する機能。
- **回答送信機能:** 幹事が提示した日程候補 に対する都合やコメント、自身の希望予算、希望エリア、全体コメント をデータベースに保存する送信機能。
- **回答の更新（再送信）機能:** 一度登録した回答内容を上書き更新して再送信する機能。
- **確定内容の取得機能:** 幹事によって決定された確定内容（日時、予算、エリア、店名、払先リンク）を取得し表示するための機能。

データベース案

- users

Column Name	Type	Description
user_id (PK)	VARCHAR	ユーザID
password_hash	VARCHAR	パスワード（ハッシュ値）
display_name	VARCHAR	名前（表示名）

- events

Column Name	Type	Description
event_id (PK)	INT	イベントID
manager_id (FK)	VARCHAR	管理者ID
event_name	VARCHAR	イベント名
response_deadline	DATETIME	回答締め切り日時
description	TEXT	コメント（詳細）
status	ENUM/VARCHAR	ステータス ('planning', 'confirmed', 'completed')
confirmed_area_id (FK)	VARCHAR	確定エリア
confirmed_shop_name	VARCHAR	確定店名
confirmed_budget	INT	確定予算
confirmed_datetime	DATETIME	確定日時
payment_destination	TEXT	支払先情報
paypay_link	VARCHAR	PayPayリンク

- event_date_candidates

Column Name	Type	Description
date_candidate_id (PK)	INT	日時候補ID
event_id (FK)	INT	イベントID
proposed_datetime	DATETIME	候補日時
total_score	INT	合計点数

- event_area_candidates

Column Name	Type	Description
area_candidate_id (PK)	INT	エリア候補ID
event_id (FK)	INT	イベントID
proposed_area	VARCHAR	候補エリア

- event_participants

Column Name	Type	Key	Description
user_id	VARCHAR	PK, FK	参加者のユーザID
event_id	INT	PK, FK	対象のイベントID
preferred_budget	INT	-	その人の希望予算
preferred_area	VARCHAR	-	その人の希望エリア
overall_comment	TEXT	-	幹事への全体メッセージ（例：「楽しみ！」）

- participant_date_responses

Column Name	Type	Key	Description
response_id	INT	PK	回答レコード固有のID
event_id	INT	FK	対象のイベントID
user_id	VARCHAR	FK	回答したユーザのID
date_candidate_id	INT	FK	幹事が作成した「日時候補」のID
score	INT	-	都合（例：5点=○, 3点=△, 0点=×）
comment	TEXT	-	その日に関する個別メモ（例：「少し遅れます」）

関数一覧

1. 認証・ユーザー管理機能

対象テーブル: ユーザ, イベント, イベント参加者

関数名	引数 (Input)	戻り値 (Output)	処理概要 (DB操作)
registerUser	ユーザID, パスワード, 名前	ユーザ 情報	ユーザ テーブルに新規レコードを INSERT する。パスワードはハッシュ化して保存する。
loginUser	ユーザID, パスワード	認証トークン, ユーザ 情報	ユーザ テーブルを ユーザID で SELECT し、ハッシュ値と照合する。
searchUsers (今は実装なし)	検索キーワード	ユーザ[]	ユーザ テーブルの 名前 または ユーザID に対して部分一致検索 (LIKE 等) で SELECT する。
getPastRecipients	自身のユーザID	ユーザ[]	イベント テーブル (管理者ID = 自分) と イベント参加者 テーブルを JOIN し、過去に招待した ユーザID を抽出。さらに ユーザ テーブルと結合して名前を取得する。

- registerUser

戻り値

成功時

```
{
  "status": "success",
  "data": {
    "token": "eyJhbGciOiJIUzI1Ni...",
    "user": { "userId": "user_001", "userName": "田中 太郎" }
  }
}
```

失敗時

```
{
  "status": "error",
  "message": "ユーザ登録時にエラーが発生しました。すでに存在するユーザIDの可能性あります。"
}
```

- loginUser

戻り値

成功時

```
{
  "status": "success",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": { "userId": "user_001", "userName": "田中 太郎" }
  }
}
```

失敗時

```
{
  "status": "error",
  "message": "ユーザIDかパスワードのどちらかが違います"
}
```

- searchUsers

成功時

```
{
  "status": "success",
  "data": {
    "users": [
      { "userId": "user_002", "userName": "佐藤 花子" },
      { "userId": "user_003", "userName": "鈴木 一郎" }
    ],
    "totalCount": 2
  }
}
```

失敗時

```
{
  "status": "error",
  "message": "ユーザ情報の取得に失敗しました"
}
```

- getPastRecipient

成功時

```
{
  "status": "success",
  "data": {
    "users": [
      { "userId": "user_004", "userName": "高橋 次郎" },
      { "userId": "user_005", "userName": "伊藤 美咲" }
    ],
    "totalCount": 2
  }
}
```

失敗時

```
{
  "status": "error",
  "message": "過去の送信先情報の取得に失敗しました"
}
```

2. 幹事向け機能（イベント管理）

対象テーブル: イベント, イベント希望日時（管理者用）, イベント希望エリア（管理者用）, イベント参加者, イベント希望日時（参加者用）

関数名	引数 (Input)	戻り値 (Output)	処理概要（DB操作）
createNewEvent	管理者ID, イベント名, コメント, 回答締め切り, 候補日時[], 候補エリア[]	イベントID	1. イベント に INSERT（ステータスは「検討中」）。 2. イベント希望日時（管理者用） に候補日時の数だけ INSERT。 3. イベント希望エリア（管理者用） に候補エリアの数だけ INSERT。
getManagedEvents	管理者ID	イベント[]	イベント テーブルを 管理者ID で SELECT し、自身が作成したイベント一覧を取得する。

inviteUsers	イベントID, 招待ユーザID[]	成功ステータス	イベント参加者 テーブルに、ユーザID と イベントID のペアを INSERT する（希望予算などはNULL/空とする）。
getEventDetail	イベントID	集計結果オブジェクト	1. イベント希望日時（参加者用）の点数を合算し、イベント希望日時（管理者用）の点数を UPDATE する。 2. イベント参加者 テーブルから 希望予算、希望エリア、全体コメントを SELECT して集計する。
confirmEvent	イベントID, 確定情報（日時, エリア, 店名, 予算, 支払先, PayPayリンク）	更新済 イベント 情報	イベント テーブルを UPDATE し、各確定項目の値を格納する

- createEvent

成功時

```
{
  "status": "success",
  "data": {
    "eventId": 101,
    "eventName": "新年会2026",
    "deadlineForResponses": "2026-02-27 15:00",
    "managerId": "幹事 太郎",
    "description": "今年の新年会です。ぜひご参加を！",
    "status": "検討中",
    "candidateDates": [
      { "id": 1, "datetime": "2026-04-01 19:00", "totalScore": 0 },
      { "id": 2, "datetime": "2026-04-02 19:00", "totalScore": 0 }
    ],
    "candidateAreas": ["新宿", "渋谷", "池袋"]
  }
}
```

失敗時

```
{
  "status": "error",
  "message": "イベントを作成できませんでした"
}
```

- getManagedEvents

成功時

```
{
  "status": "success",
  "data": {
    "underConsideration": [
      {
        "eventId": 101,
        "eventName": "新年会2026",
        "status": "検討中",
        "deadlineForResponses": "2026-02-27 15:00"
      }
    ],
    "confirmed": [
      {
        "eventId": 103,
        "eventName": "プロジェクト打ち上げ",
        "datetime": "2025-12-15 18:30",

```

```

        "status": "情報確定"
      }
    ],
    "alreadyHeld": [
      {
        "eventId": 100,
        "eventName": "忘年会2025",
        "status": "開催済み",
        "datetime": "2025-12-20 18:30",
        "shopName": "居酒屋 魚民"
      }
    ]
  }
}

```

失敗時

```

{
  "status": "error",
  "message": "イベントの取得に失敗しました"
}

```

- inviteUsers

成功時

```

{
  "status": "success",
  "data": {
    "invitedUsers": [
      { "userId": "user_002", "userName": "佐藤 花子" },
      { "userId": "user_003", "userName": "鈴木 一郎" }
    ],
    "event": {
      "eventId": 101,
      "eventName": "新年会2026"
    }
  }
}

```

失敗時

```

{
  "status": "error",
  "message": "ユーザの招待に失敗しました"
}

```

- getEventDetail

成功時

```

{
  "status": "success",
  "data": {
    "eventId": 101,

```

```

"eventName": "新年会2026",
"deadlineForResponses": "2026-02-27 15:00",
"totalParticipants": 8,
"replyCount": 6,

"dateSummaries": [
  {
    "dateCandidateId": 1,
    "datetime": "2026-04-01 19:00",
    "totalScore": 25,
    "details": { "5点": 4, "3点": 1, "0点": 1 }
  },
  {
    "dateCandidateId": 2,
    "datetime": "2026-04-05 18:30",
    "totalScore": 18,
    "details": { "5点": 2, "3点": 2, "0点": 2 }
  }
],

"areaPreferences": [
  { "area": "新宿", "count": 4 },
  { "area": "渋谷", "count": 2 }
],

"budgetSummary": {
  "average": 5500,
  "min": 4000,
  "max": 8000
},

"participantResponses": [
  {
    "userId": "user_001",
    "userName": "田中 太郎",
    "preferredBudget": 5000,
    "preferredArea": "新宿",
    "overallComment": "楽しみにしてます！禁煙席だと嬉しいです。",
    "dateResponses": [
      { "dateCandidateId": 1, "score": 5, "comment": "いつでもOK" },
      { "dateCandidateId": 2, "score": 0, "comment": "この日は出張" }
    ]
  }
],

"confirmedDetails": {
  "datetime": "2026-04-01 19:00",
  "area": "新宿",
  "shopName": "和食処 ますだ",
  "budget": 5500,
  "paymentDest": "三菱UFJ銀行 ○○支店 普通 1234567 マスダシュゾウ",
  "paypayLink": "https://paypay.me/xxxx/5500",
  "managerComment": "点数が一番高かったこの日程で予約しました！楽しみましょう。"
}
}
}

```

失敗時

```
{
  "status": "error",
  "message": "集計データを取得できませんでした"
}
```

- confirmEvent

成功時

```
{
  "status": "success",
  "message": "イベントを確定しました。参加者に詳細情報が公開されます。",
  "data": {
    "eventId": 101,
    "eventName": "新年会2026",
    "deadlineForResponses": "2026-02-27 15:00",
    "status": "確定済み",
    "managerId": "user_001",
    "updatedAt": "2026-03-01T16:45:00Z",

    "confirmedDetails": {
      "datetime": "2026-04-01 19:00",
      "area": "新宿",
      "shopName": "和食処 ますだ",
      "budget": 5500,
      "paymentDest": "三菱UFJ銀行 ○○支店 普通 1234567 マスダシュゾウ",
      "paypayLink": "https://paypay.me/xxxx/5500",
      "managerComment": "点数が一番高かったこの日程で予約しました！楽しみましょう。"
    },

    "summary": {
      "totalParticipants": 8,
      "replyCount": 8
    }
  }
}
```

失敗時

```
{
  "status": "error",
  "message": "データの確定に失敗しました"
}
```

3. 参加者向け機能（回答管理）

対象テーブル: イベント, イベント参加者, イベント希望日時 (参加者用), イベント希望日時 (管理者用)

関数名	引数 (Input)	戻り値 (Output)	処理概要 (DB操作)
getParticipatingEvents	自身のユーザID, フィルタ状態 (未回答/回答済)	イベント[]	イベント参加者 テーブルを ユーザID で SELECT し、イベント テーブルと結合。 ※ 希望予算 等がNULL (または イベント希望日時 (参加者用) に紐づくレコードがない) 場合は「未回答」と判定してフィルタリングする。

<code>getEventDetails</code>	イベントID	イベント詳細オブジェクト	イベント、イベント希望日時（管理者用）、イベント希望エリア（管理者用）を <code>SELECT</code> し、幹事が提示した候補情報を表示する。
<code>submitResponse</code> (兼 <code>updateResponse</code>)	イベントID, 自身のユーザID, 回答内容（各日時の点数・コメント, 予算, エリア, 全体コメント）	成功ステータス	1. イベント参加者 テーブルの 希望予算、希望エリア、全体コメント を <code>UPDATE</code> する。 2. イベント希望日時（参加者用） テーブルに、各 イベント希望日時ID に対する 点数 と コメント を <code>UPSERT</code> （存在しなければ <code>INSERT</code> 、あれば <code>UPDATE</code> ）する。
<code>getConfirmedDetails</code>	イベントID	確定情報オブジェクト	イベント テーブルを <code>SELECT</code> し、イベントステータスが「開催済み」の場合のみ、エリア・店名・予算・日時・支払先・PayPayリンクを返す。

- `getParticipatingEvents`

成功時

```
{
  "status": "success",
  "data": {
    "unanswered": [
      {
        "eventId": 101,
        "eventName": "新年会2026",
        "deadlineForResponses": "2026-03-10 15:00",
        "managerName": "幹事 太郎"
      }
    ],
    "answered": [
      {
        "eventId": 102,
        "eventName": "28卒同窓会",
        "deadlineForResponses": "2026-03-15 17:00",
        "managerName": "幹事 次郎"
      },
      {
        "eventId": 99,
        "eventName": "忘年会2025",
        "datetime": "2025-12-01 19:00",
        "managerName": "幹事 太郎"
      }
    ]
  }
}
```

失敗時

```
{
  "status": "error",
  "message": "イベントを取得できませんでした"
}
```

- `getEventDetails`

成功時

```
{
  "status": "success",
```

```

"data": {
  "eventId": 101,
  "eventName": "新年会2026",
  "managerName": "hanyunaoki",
  "description": "今年の新年会です。希望の日程とエリアを教えてください！",

  "candidateDates": [
    { "dateCandidateId": 1, "datetime": "2026-04-01 19:00" },
    { "dateCandidateId": 2, "datetime": "2026-04-05 18:30" }
  ],
  "candidateAreas": ["新宿", "渋谷", "池袋"]
}
}

```

失敗時

```

{
  "status": "error",
  "message": "イベントの取得に失敗しました"
}

```

- submitResponse

成功時

```

{
  "status": "success",
  "message": "回答を保存しました",
  "data": {
    "eventId": 101,
    "eventName": "新年会2026",

    "dateResponses": [
      { "dateCandidateId": 1, "datetime": "2026-04-01 19:00", "score": 5, "comment": "いつでもOK" },
      { "dateCandidateId": 2, "datetime": "2026-04-05 18:30", "score": 0, "comment": "出張のため不可" }
    ],
    "preferredBudget": 5000,
    "preferredArea": "新宿",
    "overallComment": "楽しみにしてます！"
  }
}

```

失敗時

```

{
  "status": "error",
  "message": "メッセージの送信に失敗しました"
}

```

- getConfirmedDetails

成功時

```
{
  "status": "success",
  "data": {
    "eventId": 101,
    "eventName": "新年会2026",
    "managerName": "hanyunaoki",

    "confirmedDetails": {
      "datetime": "2026-04-01 19:00",
      "area": "新宿",
      "shopName": "和食処 ますだ",
      "budget": 5500,
      "paymentDest": "三菱UFJ銀行 ○○支店 普通 1234567 マスダシュゾウ",
      "paypayLink": "https://paypay.me/xxxx/5500",
      "managerComment": "点数が一番高かったこの日程で予約しました！楽しみましょう。"
    }
  }
}
```

失敗時

```
{
  "status": "error",
  "message": "イベントの取得に失敗しました"}
```

クラス一覧

クラス名	保持するプロパティ (DBのカラムそのまま)	備考
User	user_id, name	基本となるユーザー情報
Event	event_id, manager_id, name, response_deadline, comment, status, area, shop_name, budget, datetime, payment_dest, paypay_link	確定情報もここに保持
EventDateCandidate	id, event_id, datetime, total_score	幹事が設定した候補日と集計点数
EventAreaCandidate	id, event_id, area	幹事が設定した候補エリア
Participant	user_id, event_id, preferred_budget, preferred_area_id, comment	参加者ごとの希望条件
ParticipantResponse	id, event_id, user_id, date_candidate_id, comment, score	日時候補1つ1つに対する回答
EventDetail	event: Event dateCandidates: EventDateCandidate[] areaCandidates: EventAreaCandidate[]	イベントの基本情報に、候補日とエリアのリストをまとめたもの
EventSummary	event: Event participants: Participant[]、 dateSummaries: EventDateCandidate[], budgetStats: (平均、最小、最大など	幹事が見る集計結果。計算結果や、参加者全員の回答をまとめたもの

データベース操作関数一覧

1. 認証・ユーザー管理系

機能名 (JP)	英語関数名 (EN)	操作内容 (SQL)	対象テーブル	備考	引数	戻り値
ユーザー登録	createUser	INSERT	users	パスワードはハッシュ化して保存。	id, password, displayName	User クラスインスタンス

パスワード照会	<code>findPasswordHashById</code>	SELECT	<code>users</code>	ログイン時に利用	id	パスワード（ハッシュ化されたもの）
ユーザー取得	<code>findUserById</code>	SELECT	<code>users</code>	パスワード照会してから利用	id	<code>User</code> クラスインスタンス
ユーザー検索(いない、フロントエンド側に一任する)	<code>findUsersByKeyword</code>	SELECT	<code>users</code>	<code>LIKE %keyword%</code> 。セキュリティ上、ID完全一致のみにする案もあり。		
過去の送信先取得	<code>findPastRecipients</code>	SELECT + JOIN	<code>users</code> , <code>events</code> , <code>event_participants</code>	自身が管理者のイベントから参加者を抽出。	id	<code>User</code> クラスインスタンス配列
名前照会(いないかも)	<code>inquiryUserName</code>	SELECT	<code>users</code>	idから名前を抽出	id	<code>User</code> クラスインスタンス
id照会	<code>findIdByName</code>	SELECT	<code>users</code>	名前からIDを抽出	name	<code>User</code> クラスインスタンス

2. 幹事向け（イベント管理）系

機能名 (JP)	英語関数名 (EN)	操作内容 (SQL)	対象テーブル	備考	引数
イベント基本作成	<code>createEvent</code>	INSERT	<code>events</code>	ステータスを <code>planning</code> （検討中）で作成。	<code>manager_id</code> , <code>ev</code>
候補日時登録	<code>bulkInsertDateCandidates</code>	INSERT (複数)	<code>event_date_candidates</code>	候補日時の数だけループでレコード作成。	<code>{event_id, prc</code>
候補エリア登録	<code>bulkInsertAreaCandidates</code>	INSERT (複数)	<code>event_area_candidates</code>	候補エリアの数だけループでレコード作成。	<code>{event_id, prop</code>
管理イベント一覧取得	<code>findEventsByManagerId</code>	SELECT	<code>events</code>	<code>manager_id</code> が自分のIDのものを取得。	<code>manager_id</code>
招待送信 (紐付け)	<code>bulkInsertParticipants</code>	INSERT (複数)	<code>event_participants</code>	選択したユーザーをイベントに登録。	<code>{event_id, user.</code>
点数集計更新	<code>updateDateCandidateScores</code>	SUM → UPDATE	<code>participant_date_responses</code> → <code>event_date_candidates</code>	参加者の合計点数を計算し、マスター側を更新。	<code>event_id</code>
回答詳細集計	<code>findResponseSummaryByEventId</code>	SELECT	<code>event_participants</code>	全参加者の希望予算、エリア、コメントを取得。	<code>event_id</code>
イベント確定	<code>updateEventAsConfirmed</code>	UPDATE	<code>events</code>	ステータスを <code>confirmed</code> にし、情報を一括更新。	<code>event_id</code> , <code>conf</code> <code>confirmed_shoj</code> <code>confirmed_date</code> <code>paypay_link(nul</code>

3. 参加者向け（回答管理）系

機能名 (JP)	英語関数名 (EN)	操作内容 (SQL)	対象テーブル	備考	引数
参加イベント一覧取得	<code>findEventsByParticipantId</code>	SELECT + JOIN	<code>event_participants</code> , <code>events</code>	自分が招待されているイベントを結合して取得。	<code>user_id</code>
イベント詳細取得(いない、幹	<code>findEventDetailById</code>	SELECT	<code>events</code> , <code>event_date_candidates</code> ,	幹事が提示した条件一式を取	<code>event_id</code>

事例の処理)			event_area_candidates	得。	
参加者基本回答	updateParticipantBaseResponse	UPDATE	event_participants	希望予算、希望エリア、全体コメントを保存。	event_id, user_id, budget, area, comment
日程別回答 (UPSERT)	upsertDateResponses	INSERT or UPDATE	participant_date_responses	既存回答があれば更新、なければ新規作成。	event_id, user_id, budget, area, comment
確定内容取得(いない)	getConfirmedDetails	SELECT	events	ステータスが confirmed または completed の情報を取得。	